

PROJECT DOCUMENTATION

IMPLEMENT CLIENT SCRIPT & UI POLICY

(INCIDENT)

TECHNOLOGY TRACK

ServiceNow Administrator

TEAM DETAILS

TeamID:SWTID-2026- 3508

Team Size:5

Team Leader:

DEEPIKARANI. P

Team Members:

INDHUMATHI. K S

BHAVADHARANI.D

ELAKKIYA. K

HARINI. M

Bon Secours Arts and Science College for Women

Mannargudi

PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

1. Introduction

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow can use UI Policies and Client Scripts to control Incident form behavior.

Incident records need accurate and complete information for proper triage, routing, resolution, reporting, and SLA compliance. Manual data entry can result in incomplete or incorrect information. This project addresses that problem by applying conditional field behavior and validation at the user-interface level.

2. Problem Statement

When users create or update Incident records, they may leave required fields empty or enter inconsistent information. This can affect data accuracy, incident routing, reporting, SLA compliance, and service quality.

Therefore, UI Policies and Client Scripts are implemented to enforce business rules and validate Incident records before submission.

3. Objective

The main objective of this project is to demonstrate how ServiceNow client-side controls can maintain data integrity in Incident records.

4. Technologies / Skills Used

- ServiceNow
- Incident Management
- UI Policy
- UI Policy Actions
- Client Scripts
- Form Validation
- JavaScript

5. Project Implementation

Task 1: Create UI Policy on Incident

AUIPolicy named HighImpactControl is created for the Incident table.

Condition: Field = Impact; Operator = is; Value = 1 – High.

When the Incident Impact is High, the UI Policy is triggered. The Assignment Group field is configured as mandatory, and Reverse if false is enabled so that the changes are reversed when the condition is no longer true.

Task 2: Create UI Policy Action – Urgency

A UI Policy Action is configured for the Urgency field.

Configuration: Field = Urgency; Read-only = True; Visible = unchanged.

When Impact is High, the Urgency field becomes read-only, preventing users from manually changing it.

Task 3: Create onChange Client Script

AnonChange Client Script named Autoset urgency for high impact is created for the Incident table. It runs whenever the Impact field changes. When Impact is High, the script automatically sets Urgency to High.

Script

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage(
            'Urgency set to High for High impact incident.'
        );
    }
}
```

Task 4: Create onSubmit Client Script

AnonSubmit Client Script named Prevent save if Assigned To missing is created. Its purpose is to prevent saving a High Impact Incident when the Assigned To field is empty.

Script

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorBox(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );

        return false;
    }

    return true;
}
```

```
}
```

Task 5: Create onCellEdit Client Script

AnonCellEdit ClientScript is created for the State field. Its purpose is to prevent users from changing the Incident State directly from the list view.

Script

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);  
}
```

6. Testing

Test 1: Mandatory Enforcement

An Incident is created with Impact=High while leaving Assigned To empty.

Expected Result: The Incident should not be saved. An error message should be displayed, and configured UI Policy and Client Script behaviors should work correctly.

Test 2: Successful Save

The Assigned To field is filled and the Incident is submitted.

Expected Result: The Incident should save successfully. Urgency auto-setting, read-only behavior, and State restrictions should work.

Test 3: Reverse Condition

Impact is changed from High to Medium.

Expected Result: Assigned To should no longer be mandatory, Urgency should become editable, and the Incident should save successfully.

Test 4: List Edit Blocking

The State field is edited directly from the Incident list.

Expected Result: An alert message appears and the State value remains unchanged.

Test 5: Form-Based Update

The State field is changed from the Incident form instead of the list.

Expected Result: The State change should save successfully and other UI Policy and Client Script rules should continue working.

7. Benefits

- Improves Incident data accuracy.
- Ensures required information is provided.
- Automatically updates fields based on conditions.
- Prevents invalid record submission.
- Controls unwanted list-based modifications.
- Improves consistency of Incident records.
- Provides better user interaction with Incident forms.

8. Conclusion

The Implement Client Script & UI Policy (Incident) project demonstrates how UI Policies and Client Scripts can work together to control Incident form behavior. The implementation uses onChange, onSubmit, and onCellEdit Client Scripts along with UI Policy Actions to maintain data integrity.

The project ensures that fields such as Assigned To and Urgency respond appropriately to Impact changes and that incorrect submissions or unauthorized list edits are prevented. The solution is lightweight, efficient, and suitable for a ServiceNow micro project.